

✓ Name: Prathamesh Arvind Jadhav

Roll No: 3014

Experiment No: 6:- Designing and developing model for Text generation using LSTM.

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Embedding
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Step 2: Prepare the Text Data
text = """Prathamesh Jadhav is a passionate AI and Data Science graduate with a strong foundation in programming, data
He has hands-on experience in developing machine learning models, building intelligent web applications, and working w
With expertise in Python, SQL, C++, and popular frameworks like Flask and Streamlit, he delivers efficient and scalabl
He has successfully completed projects in placement prediction, air quality forecasting, and intelligent learning plat
Known for his teamwork, problem-solving, and eagerness to explore cutting-edge technologies, he aims to drive innovati

# Step 3: Tokenize the Text
tokenizer = Tokenizer()
tokenizer.fit_on_texts([text])
total_words = len(tokenizer.word_index) + 1

# Step 4: Generate Input Sequences
input_sequences = []
for line in text.split('.'):
    token_list = tokenizer.texts_to_sequences([line])[0]
    for i in range(1, len(token_list)):
        n_gram_sequence = token_list[:i+1]
        input_sequences.append(n_gram_sequence)

# Step 5: Pad Sequences
max_sequence_len = max([len(x) for x in input_sequences])
input_sequences = np.array(pad_sequences(input_sequences, maxlen=max_sequence_len, padding='pre'))

# Step 6: Split Input and Output
X = input_sequences[:, :-1]
labels = input_sequences[:, -1]
y = tf.keras.utils.to_categorical(labels, num_classes=total_words)

# Step 7: Build the LSTM Model
model_textgen_lstm = Sequential()
model_textgen_lstm.add(Embedding(total_words, 64, input_length=max_sequence_len - 1))
model_textgen_lstm.add(LSTM(100))
model_textgen_lstm.add(Dense(total_words, activation='softmax'))

# Step 8: Compile and Train the Model
model_textgen_lstm.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

```
model_textgen_lstm.fit(X, y, epochs=100, verbose=1)
```

```

3/3 ————— 0s 28ms/step - accuracy: 0.8937 - loss: 1.1394
Epoch 73/100
3/3 ————— 0s 26ms/step - accuracy: 0.9171 - loss: 1.0890
Epoch 74/100
3/3 ————— 0s 28ms/step - accuracy: 0.8680 - loss: 1.1630
Epoch 75/100
3/3 ————— 0s 27ms/step - accuracy: 0.8875 - loss: 1.1102
Epoch 76/100
3/3 ————— 0s 26ms/step - accuracy: 0.8797 - loss: 1.0959
Epoch 77/100
3/3 ————— 0s 26ms/step - accuracy: 0.9360 - loss: 0.9658
Epoch 78/100
3/3 ————— 0s 26ms/step - accuracy: 0.8891 - loss: 1.0267
Epoch 79/100
3/3 ————— 0s 27ms/step - accuracy: 0.9087 - loss: 1.0238
Epoch 80/100
3/3 ————— 0s 25ms/step - accuracy: 0.9048 - loss: 0.9708
Epoch 81/100
3/3 ————— 0s 25ms/step - accuracy: 0.9282 - loss: 0.9866
Epoch 82/100
3/3 ————— 0s 26ms/step - accuracy: 0.9243 - loss: 0.9350
Epoch 83/100
3/3 ————— 0s 27ms/step - accuracy: 0.7555 - loss: 1.1296
Epoch 84/100
3/3 ————— 0s 26ms/step - accuracy: 0.8441 - loss: 1.0215
Epoch 85/100
3/3 ————— 0s 26ms/step - accuracy: 0.9060 - loss: 0.9529
Epoch 86/100
3/3 ————— 0s 26ms/step - accuracy: 0.8725 - loss: 0.9861
Epoch 87/100
3/3 ————— 0s 26ms/step - accuracy: 0.9087 - loss: 0.9509
Epoch 88/100
3/3 ————— 0s 32ms/step - accuracy: 0.9377 - loss: 0.9109
Epoch 89/100
3/3 ————— 0s 27ms/step - accuracy: 0.9338 - loss: 0.8446
Epoch 90/100
3/3 ————— 0s 29ms/step - accuracy: 0.9299 - loss: 0.8736
Epoch 91/100
3/3 ————— 0s 26ms/step - accuracy: 0.9181 - loss: 0.8493
Epoch 92/100
3/3 ————— 0s 26ms/step - accuracy: 0.9416 - loss: 0.8095
Epoch 93/100
3/3 ————— 0s 26ms/step - accuracy: 0.9181 - loss: 0.8516
Epoch 94/100
3/3 ————— 0s 26ms/step - accuracy: 0.9142 - loss: 0.8475
Epoch 95/100
3/3 ————— 0s 26ms/step - accuracy: 0.9416 - loss: 0.7611
Epoch 96/100
3/3 ————— 0s 34ms/step - accuracy: 0.9237 - loss: 0.7995
Epoch 97/100
3/3 ————— 0s 28ms/step - accuracy: 0.9181 - loss: 0.7673
Epoch 98/100
3/3 ————— 0s 27ms/step - accuracy: 0.9198 - loss: 0.7723
Epoch 99/100
3/3 ————— 0s 26ms/step - accuracy: 0.9488 - loss: 0.7599
Epoch 100/100
3/3 ————— 0s 26ms/step - accuracy: 0.9198 - loss: 0.7422
<keras.src.callbacks.history.History at 0x7a443e4e6910>

```

```
# Step 9: Generate New Text
```

```

def generate_text(seed_text, next_words, max_sequence_len):
    for _ in range(next_words):
        token_list = tokenizer.texts_to_sequences([seed_text])[0]
        token_list = pad_sequences([token_list], maxlen=max_sequence_len-1, padding='pre')
        predicted = np.argmax(model_textgen_lstm.predict(token_list, verbose=0), axis=-1)
        output_word = ""
        for word, index in tokenizer.word_index.items():
            if index == predicted:
                output_word = word
                break

```

```
        output_word = word
        break
    seed_text += " " + output_word
    return seed_text
```

Example: Generating Text

```
print(generate_text("Prathamesh Jadhav is a passionate AI and Data Science graduate with a strong foundation in programn
```

➞ Prathamesh Jadhav is a passionate AI and Data Science graduate with a strong foundation in programming, data analy

